

```
In [1]: import pandas as pd
import yfinance as yf
import numpy as np
```

```
In [2]: def ler_df_caminho(caminho):
    df = pd.read_csv(caminho)
    df["Data"] = pd.to_datetime(df["Data"])
    cols_info = ["Mes", "Data", "Modelo"]
    tickers = [c for c in df.columns if c not in cols_info]
    datas = df["Data"].sort_values().tolist()

    inicio = datas[0]
    fim = datas[-1] + pd.Timedelta(days=40)

    # 2. Baixar preços das ações
    precos = yf.download(tickers, start=inicio, end=fim)["Close"]

    if isinstance(precos, pd.Series):
        precos = precos.to_frame()

    # 4. Expandir pesos ao longo do tempo
    pesos_diaros = pd.DataFrame(0.0, index=precos.index, columns=tickers)
    for i, data_reb in enumerate(datas):
        linha = df[df["Data"] == data_reb]
        if linha.empty:
            raise ValueError(f"Data de rebalanceamento {data_reb.date()} não existe")
        pesos = linha[tickers].iloc[0].fillna(0) # Preencher NaN com 0

        # Próximo rebalanceamento ou final
        if i < len(datas)-1:
            prox_data = datas[i+1]
        else:
            prox_data = precos.index[-1]

        mask = (pesos_diaros.index >= data_reb) & (pesos_diaros.index < prox_data)
        pesos_diaros.loc[mask, :] = pesos.values

    # ✅ CORREÇÃO: Calcular retornos percentuais
    retornos = precos.pct_change()

    # ✅ Calcular retorno diário do portfólio (soma ponderada)
    portfolio_returns = (retornos * pesos_diaros).sum(axis=1)

    # ✅ Calcular valor acumulado começando em 100
    portfolio_close = (1 + portfolio_returns).cumprod() * 100
    portfolio_close.iloc[0] = 100 # Garantir que começa em 100

    return portfolio_close, inicio, fim, tickers
```

```
In [3]: def ler_df_portifolio(caminhos=['portfolios_mensais_RF_2024-10-01_2025-10-01.csv'],
    dados = {}
    tickers = {}
    for caminho in caminhos:
```

```

portfolio_close, inicio, fim, ticker = ler_df_caminho(caminho)
dados[caminho.split(sep='_')[2]] = portfolio_close
tickers[caminho.split(sep='_')[2]] = ticker

# 3. Baixar IBOV
ibov = yf.download("^BVSP", start=inicio, end=fim)["Close"]

#  Calcular IBOV normalizado
ibov_returns = ibov.pct_change()
ibov_close = (1 + ibov_returns).cumprod() * 100
ibov_close.iloc[0] = 100

dados['IBOV'] = ibov_close['^BVSP']
# 6. Juntar tudo
df_final = pd.DataFrame(dados).dropna()

return df_final, dados, tickers

```

```

In [4]: def compute_drawdown(series):
        cummax = series.cummax()
        dd = (series - cummax) / cummax
        max_dd = dd.min()

        # Encontrar datas do pico e vale
        valley = dd.idxmin()
        peak = series.loc[:valley].idxmax()

        return dd, max_dd, peak, valley

def compute_metrics(series, benchmark=None, rf_rate=0):
    ret = series.pct_change().dropna()

    ann_factor = 252

    # Retornos
    cumulative = series.iloc[-1] / series.iloc[0] - 1
    daily_mean = ret.mean()
    annual_return = (1 + daily_mean)**ann_factor - 1

    # Risco
    daily_vol = ret.std()
    annual_vol = daily_vol * np.sqrt(ann_factor)

    # Drawdown
    dd_series, max_dd, peak, valley = compute_drawdown(series)

    # VaR e ES
    var_95 = ret.quantile(0.05)
    var_99 = ret.quantile(0.01)
    es_95 = ret[ret <= var_95].mean()
    es_99 = ret[ret <= var_99].mean()

    # Sharpe e Sortino

```

```

downside = ret[ret < 0].std()
sharpe = (daily_mean - rf_rate/252) / daily_vol if daily_vol != 0 else np.nan
sortino = (daily_mean - rf_rate/252) / downside if downside != 0 else np.nan
calmar = annual_return / abs(max_dd) if max_dd != 0 else np.nan

# Informação vs benchmark
if benchmark is not None:
    bench_ret = benchmark.pct_change().dropna()
    aligned = ret.align(bench_ret, join="inner")
    te = (aligned[0] - aligned[1]).std() * np.sqrt(ann_factor)
    ir = (annual_return - (1 + bench_ret.mean())**252 + 1) / te if te != 0 else
    corr = aligned[0].corr(aligned[1])
    beta = aligned[0].cov(aligned[1]) / aligned[1].var()
else:
    te = ir = corr = beta = np.nan
monthly = series.resample("M").agg(["first", "last"])
sinal_mensal = (monthly["last"] > monthly["first"]).mean()
return {
    "Retorno acumulado": cumulative,
    "Retorno anualizado": annual_return,
    "Retorno médio diário": daily_mean,

    "Volatilidade diária": daily_vol,
    "Volatilidade anualizada": annual_vol,

    "Max Drawdown": max_dd,
    "DD pico": peak,
    "DD vale": valley,

    "VaR 95%": var_95,
    "ES 95%": es_95,
    "VaR 99%": var_99,
    "ES 99%": es_99,

    "Sharpe": sharpe,
    "Sortino": sortino,
    "Calmar": calmar,

    "Tracking Error": te,
    "Information Ratio": ir,
    "Correlação com IBOV": corr,
    "Beta vs IBOV": beta,
    "Acerto de sinal mensal": sinal_mensal
}

```

```

def ler_df_metricas(df_series_base_100):
    results = {}
    df = df_series_base_100
    for col in df.columns:
        if col != "IBOV":
            results[col] = compute_metrics(df[col], benchmark=df["IBOV"])
        else:
            results[col] = compute_metrics(df[col])

```

Transformar em DataFrame organizado

```
metrics_df = pd.DataFrame(results)
```

```
return metrics_df
```

```
In [5]: def main_visualizacao(caminhos=['portfolios_mensais_RF_2024-10-01_2025-10-01.csv'],  
df_final, dados, tickers = ler_df_portifolio(caminhos)  
df_metricas = ler_df_metricas(df_final)
```

```
return df_final, df_metricas, tickers
```

```
In [6]: df , df_metricas, tickers = main_visualizacao()
```

```
C:\Users\gabri\AppData\Local\Temp\ipykernel_8804\859955796.py:13: FutureWarning: YF.  
download() has changed argument auto_adjust default to True  
precos = yf.download(tickers, start=inicio, end=fim)["Close"]  
[*****100%*****] 11 of 11 completed  
C:\Users\gabri\AppData\Local\Temp\ipykernel_8804\859955796.py:13: FutureWarning: YF.  
download() has changed argument auto_adjust default to True  
precos = yf.download(tickers, start=inicio, end=fim)["Close"]  
[*****100%*****] 43 of 43 completed  
C:\Users\gabri\AppData\Local\Temp\ipykernel_8804\384148243.py:11: FutureWarning: YF.  
download() has changed argument auto_adjust default to True  
ibov = yf.download("^BVSP", start=inicio, end=fim)["Close"]  
[*****100%*****] 1 of 1 completed  
C:\Users\gabri\AppData\Local\Temp\ipykernel_8804\3934820450.py:52: FutureWarning:  
'M' is deprecated and will be removed in a future version, please use 'ME' instead.  
monthly = series.resample("M").agg(["first", "last"])  
C:\Users\gabri\AppData\Local\Temp\ipykernel_8804\3934820450.py:52: FutureWarning:  
'M' is deprecated and will be removed in a future version, please use 'ME' instead.  
monthly = series.resample("M").agg(["first", "last"])  
C:\Users\gabri\AppData\Local\Temp\ipykernel_8804\3934820450.py:52: FutureWarning:  
'M' is deprecated and will be removed in a future version, please use 'ME' instead.  
monthly = series.resample("M").agg(["first", "last"])
```

```
In [7]: df_metricas
```

Out[7]:

	RF	media3m	IBOV
Retorno acumulado	-0.010326	0.017518	0.061776
Retorno anualizado	0.002254	0.037694	0.073327
Retorno médio diário	0.000009	0.000147	0.000281
Volatilidade diária	0.009973	0.012603	0.009684
Volatilidade anualizada	0.158314	0.200059	0.153727
Max Drawdown	-0.111424	-0.156329	-0.112212
DD pico	2025-05-13 00:00:00	2025-03-19 00:00:00	2024-10-02 00:00:00
DD vale	2025-08-21 00:00:00	2025-05-07 00:00:00	2025-01-03 00:00:00
VaR 95%	-0.014881	-0.019143	-0.014109
ES 95%	-0.019832	-0.028618	-0.020057
VaR 99%	-0.020763	-0.03593	-0.025509
ES 99%	-0.025773	-0.038784	-0.029503
Sharpe	0.000896	0.011652	0.029001
Sortino	0.001546	0.017588	0.045415
Calmar	0.020232	0.241118	0.653463
Tracking Error	0.114907	0.166559	NaN
Information Ratio	-0.61852	-0.213935	NaN
Correlação com IBOV	0.729169	0.58388	NaN
Beta vs IBOV	0.750924	0.759855	NaN
Acerto de sinal mensal	0.538462	0.461538	0.538462

In [8]: tickers

```
Out[8]: {'RF': ['LREN3.SA',
               'WEGE3.SA',
               'RDOR3.SA',
               'EMBR3.SA',
               'PETR4.SA',
               'ABEV3.SA',
               'SUZB3.SA',
               'RAIL3.SA',
               'ELET3.SA',
               'VALE3.SA',
               'PRIO3.SA'],
         'media3m': ['TRAD3.SA',
                    'LREN3.SA',
                    'PSSA3.SA',
                    'VIVT3.SA',
                    'OIBR3.SA',
                    'DIRR3.SA',
                    'EMBR3.SA',
                    'CSNA3.SA',
                    'TEND3.SA',
                    'SUZB3.SA',
                    'PLPL3.SA',
                    'TRIS3.SA',
                    'PETZ3.SA',
                    'VLID3.SA',
                    'GGBR4.SA',
                    'PETR4.SA',
                    'CMIG4.SA',
                    'MSFT34.SA',
                    'EMAE4.SA',
                    'SLCE3.SA',
                    'B3SA3.SA',
                    'AERI3.SA',
                    'TOTS3.SA',
                    'CYRE3.SA',
                    'BPAC11.SA',
                    'BBAS3.SA',
                    'EVEN3.SA',
                    'ABEV3.SA',
                    'ELET3.SA',
                    'TIMS3.SA',
                    'GFSA3.SA',
                    'MGLU3.SA',
                    'MOVI3.SA',
                    'PCAR3.SA',
                    'HBOR3.SA',
                    'RENT3.SA',
                    'HYPE3.SA',
                    'ASAI3.SA',
                    'TFCO4.SA',
                    'RDOR3.SA',
                    'MRVE3.SA',
                    'PINE4.SA',
                    'SBSP3.SA']}]}
```

In [9]: df

Out[9]:

	RF	media3m	IBOV
Date			
2024-10-01	100.000000	100.000000	100.000000
2024-10-02	100.600882	101.140812	100.769840
2024-10-03	99.566699	99.473539	99.378844
2024-10-04	99.942768	100.313841	99.469414
2024-10-07	98.788584	99.807149	99.639986
...	...	...	...
2025-10-06	101.365919	106.041010	108.387486
2025-10-07	100.197760	102.159831	106.687800
2025-10-08	100.321288	103.231581	107.283294
2025-10-09	98.967366	101.751762	106.953470
2025-10-10	98.967366	101.751762	106.177592

257 rows × 3 columns